



PLUGIN



COMPLETE PROJECT UNDERSTANDING GUIDE

One document that explains the whole application

Product behavior, user roles, web and mobile features, backend services, business workflows, APIs, data model, integrations, deployment, current limitations, and roadmap boundaries.

Purpose: Upload this PDF to ChatGPT or give it to a developer so they can understand the current repository before planning, debugging, reviewing, or changing it.

REPOSITORY SNAPSHOT

Commit 0064b3dd93e1

SYSTEM

EV charging discovery, booking, charging and operations

GENERATED

9 August 2026

APPLICATIONS

Spring Boot API · React website · Expo mobile app

How to use this PDF with ChatGPT

Attach this file at the start of a conversation, then describe the change or problem. It gives the model a shared map of the product, but the repository code remains the final source of truth.

Recommended first message

"Read the attached PLUGIN project guide completely. Treat implemented, partial, and roadmap features differently. Do not invent endpoints, fields, or behavior. Before proposing code changes, state which application, files, data entities, API routes, roles, and security boundaries are affected. If the guide and repository disagree, use the current repository and tell me about the difference."

Interpretation rules

- **IMPLEMENTED** means code exists in the current repository; it does not automatically mean production-ready or secure.
- **PARTIAL / STUB** means the user interface or backend flow exists, but an important real-world integration or guarantee is incomplete.
- **ROADMAP** means an idea appears in planning material and must not be described as a current feature.
- **KNOWN RISK** means the current code has a validated security, correctness, privacy, or deployment concern that should be fixed before launch.
- No password, signing key, payment secret, mail credential, database URI, cloud key, or other secret value is included here.

Snapshot warning: this guide describes commit [0064b3dd93e1](#). Future code changes may make parts stale. For high-impact work, ask ChatGPT to inspect the current files as well as this PDF.

Contents

01 Product summary and scope

02 Users, roles and permissions

03 Architecture and repository map

04 Core domain and data model

05 End-to-end business workflows

06 Website features and routes

07 Mobile app features and screens

08 Backend services and API catalog

09 Complete API catalog

10 Jobs, notifications and documents

11 External integrations

12 Configuration and local setup

13 Build and deployment

14 Testing and observability

15 Security and launch status

16 Implemented vs roadmap

17 Glossary

18 AI handoff checklist

1. Product summary and scope

PLUGIN is a three-application EV charging platform. Customers discover charging stations, reserve a connector, start and monitor charging, pay through an internal wallet, receive bills and manage their account. Administrators and station operators manage the supply side of the platform.

Web React customer and admin portal Public catalog, customer journeys, station-manager application, and the operations console.	API Spring Boot business backend Authentication, stations, bookings, sessions, wallet, billing, KYC, notifications, and administration.	App Expo/React Native customer app Mobile-first discovery, location-aware booking, charging, wallet, bills, profile, and notifications.
--	--	--

Primary product capabilities

Capability	What the system does	Main consumers
Identity and access	Email/password registration with OTP confirmation, login, Google sign-in, password recovery, profile security actions, and JWT-protected APIs.	Customer, admin, station operator
Station discovery	Lists, searches, filters, and shows stations with location, operating information, charging points, live summary, and per-kWh pricing.	Public visitors and customers
Reservations	Creates fixed-point or virtual/ETA bookings, snapshots price, prevents or resolves availability conflicts, supports modification, cancellation, reschedule requests, and no-show expiry.	Customer and operations
Charging sessions	Starts a session from a booking, tracks duration and calculated energy/cost, monitors wallet balance, completes sessions, updates point state, and produces billing records.	Customer and operations
Wallet and payments	Maintains balance and ledger; supports top-up, auto-top-up mandate, charging debit, withdrawal/refund, and paying unpaid bills through Razorpay-backed flows.	Customer and finance operations
Billing	Stores paid/unpaid bills, provides account statements and invoice downloads, and can email generated PDF invoices.	Customer and admin
Station onboarding	Collects station-manager identity, business, property, electricity, bank, capacity, and document evidence; supports review, approval/rejection, and portal credential issuance.	Applicant, admin, station operator
Operations	Dashboards and CRUD tools for stations, points, pricing, bookings, customers, sessions, bills, revenue, analytics, audit logs, KYC applications, and station managers.	Admin and station operator

What is not the current product

- It is not yet a proven production-grade charger-control network. The current charger command service is a stub and session energy is calculated from configured power and elapsed time, not trusted hardware meter

readings.

- It is not a multi-vendor roaming or charger-protocol platform; OCPP or another device protocol is not implemented in this repository.
- Route planning, availability forecasting, peer-to-peer handoff, idle-fee automation, battery-aware charging, and reward concepts in [plugin_app_future.pdf](#) are roadmap ideas only.
- The older SRS is useful background, but it predates the current MongoDB, wallet/Razorpay, mobile, and expanded KYC implementation.

2. Users, roles and permissions

Role / actor	Intended access	Main entry points
Public visitor	Landing page, station search/details, registration/login/password recovery, and beginning a station-manager application.	Website public routes and selected <code>/api/auth</code> , <code>/api/stations</code> , and KYC endpoints
CUSTOMER	Own dashboard, vehicles, bookings, charging sessions, bills, wallet, notifications, profile, and account-security actions.	Website <code>/customer/*</code> , native app, customer API groups
STATION_OPERATOR	Operate only the station or stations linked to that operator; access operational dashboard and re-KYC where appropriate.	Website <code>/admin/*</code> shell and station-scoped backend operations
ADMIN	Platform-wide operations, station/point/pricing management, customer controls, bookings/sessions/bills, analytics, audit logs, and station-manager approval.	Website <code>/admin/*</code> and <code>/api/admin/*</code>
Station-manager applicant	Submit identity/business/station/bank data and evidence, then track review status using an application reference.	Website <code>/station-manager/apply*</code> and <code>/api/station-manager/*</code>
Background scheduler	Expire bookings, send booking reminders, monitor wallet consumption, and auto-complete sessions.	Scheduled methods in backend services
External providers	Google identity, Razorpay orders/payments/refunds, SMTP email, MongoDB persistence, maps/location, and AWS/EC2 deployment.	Backend and mobile integrations

Actual authorization caveat: the website's generic `ProtectedRoute role="ADMIN"` currently treats both `ADMIN` and `STATION_OPERATOR` as accepted. Hiding sidebar links is not a security boundary. Every backend admin endpoint and every object lookup must enforce the exact role and, for operators, the linked-station scope. This is a current risk, not intended behavior.

Trust boundaries

- **Browser/mobile → API:** all IDs, roles, prices, coordinates, statuses, time windows, filenames, MIME types, and payment claims from clients are untrusted.
- **API → MongoDB:** services translate business operations into MongoDB documents; multi-document and external-payment consistency requires explicit transaction/idempotency design.
- **API → payment provider:** order creation, verification, refund, and mandate results must be authenticated and made idempotent.
- **Applicant uploads → admin viewer:** KYC files are hostile until validated, scanned, isolated, and served with safe content headers.
- **CI/CD → production:** build code must not automatically receive production secrets; deployments require least-privilege, short-lived credentials and verified hosts/artifacts.

Technology stack

Layer	Current technologies	Responsibility
Web	React 18, React Router 6, Axios, Framer Motion, Vite 5	Browser UI, local auth context, route guards, API calls
Mobile	Expo 54, React 19, React Native 0.81, AsyncStorage, Maps, Location, Notifications, Google Sign-In, Razorpay	Native customer experience and device integrations
Backend	Java 17, Spring Boot 3.2.4, Spring Web/Security/Validation/Data Mongo/Mail/Actuator, JWT, OpenPDF	Security enforcement, workflows, persistence, PDF/email, jobs
Data	MongoDB local or Atlas	Application documents, references, ledgers, KYC files and sequence counters
Delivery	Maven, npm, Android Gradle, Docker, GitHub Actions, AWS S3, EC2/systemd	Build, package and deploy

4. Core domain and data model

The backend uses MongoDB documents. Many public-facing records also receive human-friendly sequential references through `MongoSequence`. Application code—not a relational database—owns most cross-document integrity.

High-level relationships

User (role, account/profile)

- └ UserVehicle [0..many]
- └ Booking [0..many] — Station — ChargingPoint
 - └ ChargingSession [0..1 or lifecycle-linked]
 - └ Bill
- └ Wallet [1] — WalletLedgerEntry [many]
 - └ WalletTopUpAttempt [many]
- └ Notification [many]

StationManager — linked Station / operator User

- └ StationManagerApplication
 - └ StationManagerApplicationFile [standard evidence slots]
 - └ StationManagerApplicationDocument [business-specific documents]

AuditLog records important administrative activity.
PendingRegistration and PasswordResetOtp support identity workflows.
Pricing supplies the active per-kWh rate; Booking retains a pricing snapshot.

Entity catalog

Entity / collection concept	Purpose and important links
User	Identity, hashed credential, contact/profile data, role and activation state.
UserVehicle	Customer EV details used when booking and charging.
Station	Charging location, address/geography, operating information, availability and operational state.
ChargingPoint	Physical connector/charger at a station, point type/power/status, and station relationship.
Pricing	Station price definition. Current model is per kWh.
Booking	Reservation linking customer, station, optional point, vehicle, time range, location/ETA, price snapshot and status.
ChargingSession	Started charging lifecycle, timing, energy/cost progression, wallet debit progress and completion state.
Bill	Financial result of a session, amount and paid/unpaid status; source for invoice/statement output.
Wallet	Customer balance, auto-top-up settings and payment mandate details.
WalletLedgerEntry	Append-style record for credits, charging debits, refunds/withdrawals and reversals.
WalletTopUpAttempt	Razorpay order/top-up lifecycle and verification state.
Notification	Customer messages and read/unread state.

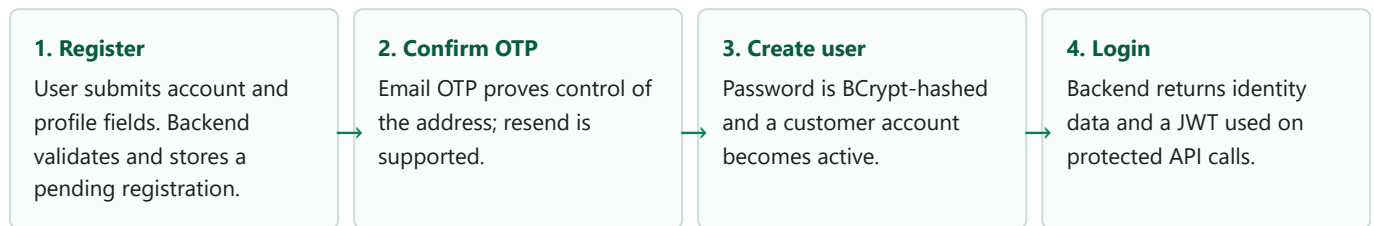
Entity / collection concept	Purpose and important links
StationManager	Approved manager/operator directory and station linkage.
StationManagerApplication	KYC/application data across personal, business, property, station, bank and capacity sections.
StationManagerApplicationFile	Binary standard evidence for government ID, selfie, property, utility, bank, installation and site slots.
StationManagerApplicationDocument	Dynamic business-type-specific document metadata and file data.
PendingRegistration	Unconfirmed registration plus OTP workflow state before creation of a full user.
PasswordResetOtp	Time-limited password recovery verification state.
AuditLog	Administrative action history for review and accountability.
MongoSequence	Counter storage for generated numeric/business references.

State and type vocabulary

Concept	Values in current code
User role	ADMIN , STATION_OPERATOR , CUSTOMER
Booking	CONFIRMED , MODIFIED , CANCELLED , COMPLETED , NO_SHOW
Reschedule request	NONE , PENDING , REJECTED ; admin approval updates the booking workflow
Charging session	IN_PROGRESS , COMPLETED , ABORTED
Bill	PAID , UNPAID
Point type / state	FAST , SLOW · AVAILABLE , RESERVED , CHARGING , OUT_OF_SERVICE , UNAVAILABLE
Pricing	PER_KWH
KYC application	PENDING , APPROVED , REJECTED
Wallet transaction	PENDING , SUCCEEDED , FAILED
Wallet ledger	MANUAL_TOP_UP , AUTO_TOP_UP , CHARGING_DEBIT , WITHDRAWAL_REFUND , REVERSAL
Applicant business	Individual, proprietorship, partnership, LLP and company-oriented document paths
Property occupancy	OWNED , RENTED

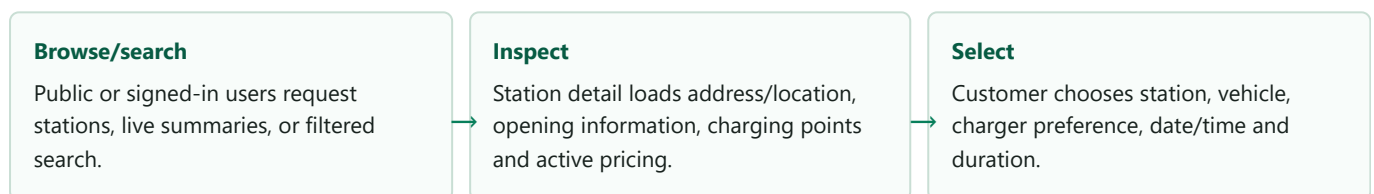
5. End-to-end business workflows

5.1 Email registration and login



Google sign-in sends a Google token to the backend, which verifies configured client identity and token/user information before selecting the account. Forgot-password supports OTP send, verify, and reset. Profile flows additionally expose change-password and account-deletion paths with authenticated and “forgot” variants. A customer may store up to three vehicles and select an active one. Deletion is blocked while future bookings, an in-progress session, or unpaid bills remain. Email OTP delivery exists; phone OTP delivery is currently a stub.

5.2 Find a charging station



5.3 Create a fixed booking

1. The customer selects a specific station and charging point, a start time, and a duration from 1 to 60 minutes.
2. The backend must confirm that the user and vehicle are eligible, the point belongs to the station, the station/point are operational, and the requested interval does not overlap another reservation.
3. The active per-kWh price is copied into the booking as a snapshot so later pricing changes do not rewrite the reservation’s commercial basis.
4. The reservation receives its reference and **CONFIRMED** status. Later actions include modification, cancellation, reschedule request, start, completion, or no-show expiry.

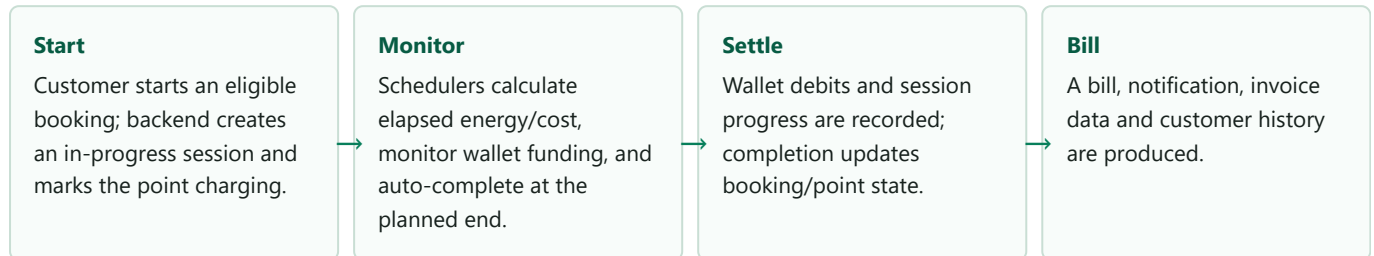
NO_SHOW exists as an enum value, but the current automatic missed-window path records **CANCELLED**.

5.4 Create a virtual/ETA booking

1. A request containing origin coordinates represents the dynamic/virtual ETA path. It can omit the physical point and carry a point-type preference. A non-ETA fixed booking may also let the backend auto-select a point, so “no point ID” alone is not the complete discriminator.
2. The backend first calculates ETA using Google Distance Matrix when configured, with a straight-line distance/speed fallback. It stores predicted arrival and a roughly 20-minute grace deadline.
3. The mobile app can update the booking’s location while the customer travels. Accepted pings are limited to about every 30 seconds; implausible speed/jumps are ignored; ETA, last-known position, ping time and grace data are updated.

4. At roughly one mile or less from the station, the service atomically changes an available point to **RESERVED** and attaches it. Allocation must also consider fixed reservation overlaps and competing virtual bookings.
5. Location updates must be permitted only for the booking owner and only for a valid virtual booking lifecycle.

5.5 Start, monitor and end charging



Current implementation boundary: `ChargerCommandService` logs commands but does not control a real charger. Energy is currently theoretical—configured power × elapsed time with an efficiency factor. Production settlement needs authenticated meter telemetry and a real device/control integration.

5.6 Wallet, Razorpay and billing

- **Manual top-up:** backend creates a Razorpay order; client completes payment; backend verifies the signed result; wallet and ledger are credited.
- **Auto-top-up:** mandate order/verification configures automatic funding; settings can be updated or disabled. A test-confirm route also exists.
- **Charging debit:** live monitoring deducts session cost and records ledger entries; this must be atomic and idempotent across instances and retries. The current default wallet starts at ₹0 with Auto-Top-Up disabled; configured threshold/amount defaults are ₹200/₹1,000 and the automatic-debit ceiling is ₹15,000.
- **Withdrawal/refund:** wallet withdrawal triggers a Razorpay refund-style external action and adjusts internal records.
- **Bill payment:** an unpaid bill can be paid from the customer wallet. Amount, ownership, balance and one-time state transition must be server-authoritative.
- **Documents:** customers can list bills, see unpaid count, download an invoice PDF, and download a date-filtered CSV account statement; admins can access operational bill/revenue views.

Current code constrains normal top-ups to ₹1,000–₹15,000 and applies a minimum final charging bill of ₹1. Deployment/provider configuration must remain consistent with these application rules.

5.7 Station-manager application and onboarding

1. Applicant requests reference data and completes sections for personal identity, business/legal registration, station/property/electricity details, bank details, charger capacity, connectors and manufacturers.
2. The multipart submission attaches standard files (government ID, selfie, registration/authorization, property, electricity bill, bank proof, installation photo, site photo) plus business-type-specific evidence.
3. The backend validates and stores the application, assigns a reference, and sends tracking communication.
4. An admin lists and reviews the application and files, then rejects with notes or approves it.

5. Credential issuance creates/links a station operator and manager directory record. A secure production design should issue a short-lived one-time setup link rather than email or display a reusable plaintext password.
6. An operator can later use the authenticated application/session endpoints and the web re-KYC route.

6. Website features and route map

The website is a Vite single-page application. `src/App.jsx` is the top-level route map; Axios wrappers under `src/api` call the backend; authentication is held by `AuthContext`; `ProtectedRoute` performs client-side navigation checks; `BillingLock` periodically blocks selected customer activity when billing rules say it should.

Public routes

Route	Screen and behavior
<code>/</code>	Landing experience and product entry point.
<code>/login</code>	Email/password and Google authentication entry.
<code>/register</code>	Customer registration and OTP confirmation journey.
<code>/forgot-password</code>	OTP-based password recovery and reset.
<code>/email-inbox</code>	Developer-style email/OTP inbox UI. Its frontend expects an email-inbox API not present in the current controller catalog; it must be removed or strictly development-gated.
<code>/search</code>	Station browsing/search/filter results.
<code>/stations/:id</code>	Station detail, charging points, status and pricing.
<code>/station-manager/apply</code>	Application introduction plus reference-based status lookup.
<code>/station-manager/apply/form</code>	Multipart station-manager KYC/application form.

Customer routes

Route	Screen and behavior
<code>/customer/dashboard</code>	Customer summary, activity and shortcuts.
<code>/customer/book/:stationId</code>	Vehicle, point/type, date/time and duration selection; submits booking data.
<code>/customer/bookings</code>	Own fixed web bookings, status/detail, cancellation, start eligibility and reschedule-request UI. The location-driven virtual flow is primarily implemented in the mobile app.
<code>/customer/sessions</code>	Active and historical charging-session status and start/end interactions.
<code>/customer/billing</code>	Bills, unpaid state, wallet payment, invoice and statement actions.
<code>/customer/profile</code>	Account/profile, password, vehicles and account-deletion controls.

Admin and operator routes

Route	Operational function
<code>/admin/dashboard</code>	Operations dashboard and summary.

Route	Operational function
/admin/re-kyc	Station-operator re-KYC form.
/admin/station-manager-applications/:id	List, inspect, preview/download files, approve/reject, and issue credentials.
/admin/station-managers	Approved station-manager directory.
/admin/stations	Create, edit, enable/disable, delete and inspect stations.
/admin/charging-points	Create/edit/delete points and change operational status.
/admin/pricing	View/create/delete station pricing records.
/admin/bookings	Platform/station bookings, stats, cancellation and reschedule decisions.
/admin/customers	Customer directory and account activation/deactivation.
/admin/sessions	Session monitoring and forced completion.
/admin/revenue	Bill and revenue reporting.
/admin/analytics	Analytics presentation derived from admin data.
/admin/audit-logs	Administrative activity review.
/admin/notifications	Admin-facing notification screen.

Screen behavior details

Area	Current web behavior
Landing	Marketing sections explain Search → Book → Charge and operator Apply → Review → Operate. A live network summary refreshes about every 30 seconds and replaces nominal station/connector statistics when data is available.
Search	Loads 12 stations per page; searches city/area/text or exactly six numeric pincode digits; All/Fast/Slow chips filter charger type. Fast/Slow filtering retrieves point data per station on the current page and refreshes about every 30 seconds.
Station detail	Loads station, points and pricing in parallel and refreshes about every 15 seconds. Booking is blocked when inactive or no point is bookable; anonymous visitors go to login and customers without a vehicle go to Profile.
Customer dashboard	Loads active sessions, recent bookings, up to 100 sessions/bills, and computes upcoming bookings, active sessions, total billed and energy metrics. Refreshes about every 30 seconds.
Web booking	Four-step fixed booking: select a usable point, choose a future local date/time, choose an integer 1–60 minute duration, review rate/details, then submit. Availability/pricing refresh about every 15 seconds and an invalid selected point is cleared.
My Bookings	Fetches up to 20 pages of 50 and paginates locally by 10; supports active-vehicle filtering, expanded details, cancellation with reason, reschedule request with date/time/reason, and session start only inside the valid window. Bookings refresh every 30 seconds and action/session state every second.
Sessions	Active session timers refresh every second and re-synchronize to server timing. Manual end requires confirmation and leads to Billing. History refreshes every 10 seconds with date/status filters and invoice download.
Billing	Shows a focused “Wallet Required” view for unpaid invoices, pays through the wallet endpoint, polls briefly for invoice finalization after session end, lists paid history, downloads authenticated PDFs, and exports CSV statements for preset/custom date ranges.
Profile/vehicles	Supports up to three vehicles, an immediately persisted active vehicle, nickname editing and deletion. Persisted make/model/registration are locked; the active vehicle cannot be removed during an active session. Typed drafts trigger leave warnings.

Area	Current web behavior
Shared locks	BillingLock checks unpaid count every second; navbar active-session state about every five seconds; notifications about every 30 seconds. Logout is blocked during an active session or unpaid-invoice state.

Admin/operator behavior details

Area	Current web behavior
Dashboard	Cards for stations, points, bookings, sessions, revenue, energy and customers. Quick actions differ visually by role.
Stations	Admin can create/delete; visible roles can edit/toggle. Form captures contact, coordinates and hours. Six-digit pincode lookup uses India Post to fill district/state.
Charging points	Select a station; add/edit/delete identifier, Fast/Slow type, connector and maximum kW; change point state with confirmation.
Pricing	Create/update/delete a per-station, per-point-type PER_KWH rule; UI prevents a price for a type not present at the selected station.
Bookings	20 rows/page, status summaries, detail, admin cancellation with reason, and pending reschedule approval/rejection. Polls when the tab is visible.
Customers	Debounce search, active/deleted filters, profile/vehicle/booking counts and reversible active-status change.
Sessions	Customer, vehicle, station, point, time, energy and status; active sessions can be force-ended after confirmation.
Revenue/analytics	Station/date-filtered revenue, sessions, energy, payment history and invoice download. Analytics is KPI cards rather than time-series charts.
Audit/notifications	Audit rows filter by entity type and show time/action/target/actor/details. A full notification route exists but the navbar bell is the normal entry.
KYC/managers	Search/filter/paginate applications, inspect all data, preview/download evidence, require notes for pending decisions, approve/reject, issue/reissue credentials, search approved managers and open their full KYC/station record.

Important browser behavior

- The Axios client adds a bearer JWT to protected requests and handles authentication failures globally.
- Current auth token and cached user profile are stored in browser **localStorage**; therefore any same-origin script injection can steal them.
- React's normal text rendering escapes strings; no routine **dangerouslySetInnerHTML** sink was found, but file previews, URLs, downloads, CSV/PDF content and server error text remain separate injection surfaces.
- Frequent polling drives billing lock, active-session status, navbar counts and operational refreshes. Production needs rate limits, deduplication, backoff and page-visibility awareness.
- Client-side guards and hidden controls improve UX only. All ownership, role, station scope, pricing and lifecycle rules must be repeated on the backend.
- The browser experience has no explicit catch-all/404 route. Web station details show address and coordinates but no interactive map.
- The web billing screen can settle an invoice from the wallet, but wallet balance, add-money, auto-top-up and Razorpay checkout management exist only in the mobile client.
- Profile notification switches are currently visual/local component state. Account-deletion code exists, but the current rendered web profile has no visible control that opens it.

7. Mobile app features and screens

The native app is customer-focused. `App.js` orchestrates authentication, global state, polling/location behavior, and an internal screen key rather than a conventional route declaration. Package identifiers use `com.plugin.mobile`.

Screen catalog

Screen / key	Customer capability
<code>AuthScreen</code>	Login, signup, OTP, forgot/reset password and native Google sign-in.
<code>home</code> · <code>HomeScreen</code>	Dashboard, quick actions and account/activity summary.
<code>stations</code> · <code>StationsScreen</code>	Station discovery, search and nearby context.
<code>stationDetails</code>	Station information, points, availability and price.
<code>stationNavigation</code>	Map/location navigation context and progress toward a station.
<code>bookingFlow</code>	Vehicle, station/charger, date/time and duration reservation steps.
<code>bookings</code>	Upcoming/current booking list and actions.
<code>bookingDetails</code>	Reference, state, schedule, station, point, ETA and lifecycle details.
<code>charging</code>	Start/current session, elapsed progress, calculated energy/cost and end action.
<code>history</code>	Previous sessions/bookings and financial history context.
<code>payment</code>	Payment workflow and Razorpay interaction.
<code>wallet</code>	Balance, ledger, top-up, auto-top-up, mandate and withdrawal-related actions.
<code>profile</code> · <code>editProfile</code>	Account display and editable profile data.
<code>vehicles</code>	Manage EVs used in booking.
<code>notifications</code>	List messages and update read state.
<code>settings</code>	App/account settings entry.
<code>changePassword</code>	Password change and OTP support.
<code>deleteAccount</code>	Authenticated or recovery-assisted account deletion flow.

Mobile behavior details

Area	Current native behavior
Startup/global	Restores JWT and compact user data from AsyncStorage, shows a short launch loader, polls unpaid invoices, enforces the Billing screen lock, refreshes many foreground screens about every four seconds, and synchronizes local booking reminders.
Home	Loads unread count, active session and up to 50 stations; requests location, sorts stations using Haversine distance, prioritizes active charging, otherwise shows the nearest charger, and links to common destinations.
Station discovery	Debounce search with All, inferred DC Fast, inferred AC, Available and Favorites filters. Details load station/points/pricing and let the user choose connector/type before reserving. Favorites stay device-local.

Area	Current native behavior
Turn-by-turn map	Uses Google-provider maps and foreground high-accuracy location, watching around every two seconds or five metres. Shows speed, heading, maneuvers, ETA/distance and arrival; recalculates after roughly 90 metres; uses Google Directions from the phone with public OSRM fallback; offers map modes, traffic, 3D tilt, overview and recenter. Arrival is around 30 metres.
Booking	Four stages—connector, schedule, review, done. Requires a saved vehicle, allows 1–60 minutes (quick 15/30/45/60), submits origin and preferred point type for the virtual flow, shows returned virtual spot, and schedules a local start reminder.
Booking details	Separates upcoming/past items, shows any active session, dynamic predicted arrival/grace/proximity lock, sends location for unlocked dynamic bookings, blocks Start until point assignment/proximity lock, and supports cancellation/reminder cleanup.
Charging	Loads/caches the active session, shows booked-end countdown, polls for backend completion/bill, supports Stop Charging, clears cache on finish and forces Billing. Offline mode can display the snapshot but cannot stop/finalize.
Bills/downloads	Lists invoices and detailed amount/energy/duration/rate data; bills are paid from Plugin Wallet. Authenticated PDFs first enter app cache; Android Storage Access Framework lets the user choose a public folder.
Wallet	Balance, withdrawable balance, ledger, native Razorpay manual top-up, threshold/amount Auto-Top-Up, card/UPI mandate, test confirmation, enable/update/disable, and refund-to-original-method withdrawal.
Account	Edit name/phone; add/select/nickname/delete vehicles; password change through current-password-plus-OTP or recovery OTP; account deletion through password-plus-OTP or recovery OTP; logout clears auth/session cache.
Notifications	Backend inbox/unread/read state plus device-local booking reminders and invoice-download notices. Notification taps can open Booking Details or Invoice Details.

Device-specific behavior

- **Location:** Expo Location and React Native Maps support nearest-station ordering and virtual-booking navigation/location updates. The current app can upload active booking location approximately every 30 seconds, so consent, minimization, expiry and owner-only access matter.
- **Offline session view:** the app caches an active-session snapshot in AsyncStorage and can show it with an offline indicator. This is a continuity display, not an authoritative offline transaction system.
- **Push/local notifications:** Expo Notifications is configured for mobile notification behavior.
- **Payments:** the native Razorpay SDK completes client payment interaction; the backend must verify all results before crediting value.
- **Downloads:** Expo File System supports invoice/PDF download behavior.
- **Network discovery:** development scripts select LAN/localhost candidates for a physical device and Metro. Current retry logic may try multiple base URLs after short timeouts.
- **Favorites:** favorite station IDs are stored only on the device; they do not synchronize with the customer account.
- **Navigation:** the app has its own in-memory tab/stack mechanism rather than React Navigation. Main destinations are Home, Stations, Charge, Bookings and Profile.

Notification boundary: the app supports backend in-app messages and device-local reminders/download notices. It does not currently register an Expo/FCM push token or implement backend remote push delivery.

Mobile network and local persistence

- The API client tries configured base/LAN origins, the Expo host, Android-emulator host, localhost/loopback and the last successful origin. Development defaults are HTTP. Normal/OTP timeouts are short (about two seconds); payment calls use a longer timeout.
- The same prepared request can currently be retried across candidates. A protected 401 or 403 clears local authentication.
- AsyncStorage keys include `plugin_token`, `plugin_user`, `plugin_api_base_url`, `plugin_active_session_snapshot`, and `plugin.favoriteStationIds`.
- JWT/user/session records are not Keystore/Keychain-backed. Android backup is enabled; logout does not erase favorites or already exported invoices. Data retention and cleanup are required.

Mobile production caveats: current builds can permit cleartext HTTP endpoints, mutation requests may retry across base URLs without idempotency, and the inspected release configuration uses a debug signing key. Before release, require HTTPS-only networking, retry only safe methods unless a server-enforced idempotency key is present, and use a protected production signing key.

8. Backend service map

Service	Responsibility
AuthService	Registration, OTP confirmation, login, Google identity and JWT-facing identity behavior.
ForgotPasswordService	Password recovery OTP send, verify and reset lifecycle.
ProfileService	Profile, vehicles, password and account-deletion operations.
StationService	Public station queries plus administrative station lifecycle.
ChargingPointService	Point inventory, station relationship and operational state.
PricingService / PricingSnapshotService	Active station price and immutable booking-time commercial snapshot.
BookingService	Reservation create/modify/cancel, overlaps, location/ETA, allocation, reschedule and expiry.
EtaService	Distance/arrival calculations used by virtual bookings.
SessionService	Session start/end, progress, auto-completion, point/booking transition and billing trigger.
ChargerCommandService	Current charger-command abstraction; implementation is logging/stub behavior.
WalletService	Balance, ledger, top-up, auto-top-up, charging debits, withdrawal/refund and live monitoring.
RazorpayPaymentService	Razorpay order, signature verification, mandate and refund HTTP/provider interaction.
BillService	Bill lookup, ownership views, wallet payment, statement and admin financial operations.
InvoicePdfService / InvoiceEmailService	OpenPDF invoice generation and email delivery.
NotificationService	Customer message creation, listing, counts and read state.
DashboardService	Admin/operator dashboard aggregations.
AdminCustomerService	Customer directory and active/inactive administration.
AuditService	Administrative audit-event storage/query.
StationOperatorAccessService	Station-operator identity/linkage and scope helper.
StationManagerApplicationService	KYC validation, create/update/status, review, approve/reject and conversion/onboarding.
StationManagerFileService	KYC binary file storage and retrieval.
StationManagerDirectoryService	Approved manager/operator directory.
StationManagerTrackingEmailService	Applicant tracking communication.
StationManagerCredentialEmailService	Current operator credential email workflow.
MongoSequenceService	Human-facing sequential IDs/references.
EntityReferenceResolver	Resolves identifiers/references across stored entities.

Backend request conventions

- Base path is normally `/api` ; the application defaults to port `8091` .
- Protected calls use an Authorization bearer JWT. Role checks in controllers/security and object-level checks in services must work together.
- JSON is used for normal request/response contracts; KYC applications use multipart form data; invoices and KYC files return binary responses.
- Validation annotations cover selected request fields, but every business invariant must also be checked in the service at the moment of mutation.
- Clients often use numeric or string IDs in paths. An authenticated user knowing an ID must never be sufficient to access or mutate another user's record.

9. Complete API catalog

This is a behavior map, not a substitute for the request/response DTO classes. “My” routes are intended to use the authenticated principal. Admin and operator scope must be enforced server-side.

Authentication — `/api/auth`

Method	Path	Purpose
POST	<code>/register</code>	Begin email registration and OTP confirmation.
POST	<code>/confirm-otp</code>	Confirm registration OTP and create/activate account.
POST / GET	<code>/resend-otp</code>	Resend registration OTP; GET is a legacy variant.
POST	<code>/login</code>	Email/password authentication.
POST	<code>/google</code>	Backend-validated Google authentication.
POST	<code>/forgot-password</code>	Legacy/combined password recovery entry.
POST	<code>/forgot-password/send-otp</code>	Send recovery OTP.
POST	<code>/forgot-password/verify-otp</code>	Verify recovery OTP.
POST	<code>/forgot-password/reset</code>	Set a new password after proof.

Stations — `/api/stations`

Method	Path	Purpose
GET	<code>/</code>	List public stations.
GET	<code>/live-summary</code>	Compact/live station availability summary.
GET	<code>/search</code>	Search/filter station catalog.
GET	<code>/{"id"}</code>	Station details.
GET	<code>/{"id"}/charging-points</code>	Points at a station.
GET	<code>/{"id"}/pricing</code>	Station price data.

Bookings — `/api/bookings`

Method	Path	Purpose
POST	<code>/</code>	Create fixed or virtual booking.
GET	<code>/my</code>	Authenticated customer’s bookings.
GET	<code>/{"id"}</code>	Single booking; must enforce owner/admin/station scope.
PUT	<code>/{"id"}</code>	Modify an eligible booking.
POST	<code>/{"id"}/cancel</code>	Cancel an eligible booking.

Method	Path	Purpose
POST	<code>/ {id} /location</code>	Update owner's active virtual-booking location/ETA.
POST	<code>/ {id} /reschedule-request</code>	Request an administrative reschedule decision.

Sessions — `/api/sessions`

Method	Path	Purpose
POST	<code>/start/ {bookingId}</code>	Start an owned, eligible booking.
POST	<code>/end/ {sessionId}</code>	End an owned session or authorized operational session.
GET	<code>/my</code>	Customer's session history.
GET	<code>/my/active</code>	Customer's current session.
GET	<code>/ {id}</code>	Single session; must enforce ownership/scope.

Bills — `/api/bills`

Method	Path	Purpose
GET	<code>/my</code>	Customer's bills.
GET	<code>/my/unpaid-count</code>	Billing lock/count indicator.
GET	<code>/ {id}</code>	Single bill; must enforce ownership/scope.
POST	<code>/my/ {id} /wallet-pay</code>	Atomically pay an owned unpaid bill from wallet.
GET	<code>/my/ {id} /invoice</code>	Download owned invoice PDF.
GET	<code>/my/statement</code>	Download customer statement.

9. Complete API catalog — continued

Wallet — `/api/wallet`

Method	Path	Purpose
GET	<code>/</code>	Authenticated customer wallet summary.
GET	<code>/ledger</code>	Wallet transaction history.
POST	<code>/topup/order</code>	Create Razorpay top-up order.
POST	<code>/topup/verify</code>	Verify payment and credit wallet once.
POST	<code>/mandate/order</code>	Create auto-top-up mandate/order.
POST	<code>/mandate/verify</code>	Verify/activate mandate.
POST	<code>/mandate/test-confirm</code>	Test-mode mandate confirmation path.
PUT	<code>/auto-topup</code>	Update auto-top-up settings.
POST	<code>/auto-topup/disable</code>	Disable automatic funding.
POST	<code>/withdraw</code>	Withdraw/refund wallet value.

Profile — `/api/profile`

Method	Path	Purpose
GET / PUT	<code>/</code>	Get or update authenticated profile.
DELETE	<code>/vehicles/{vehicleId}</code>	Delete an owned vehicle.
POST / GET	<code>/delete</code>	Account deletion; GET is legacy.
POST	<code>/delete/send-otp</code> , <code>/delete/verify-otp</code>	OTP proof for account deletion.
POST	<code>/delete/forgot/send-otp</code> , <code>/delete/forgot</code>	Recovery-assisted deletion.
POST	<code>/change-password</code>	Authenticated password change.
POST	<code>/change-password/send-otp</code> , <code>/change-password/verify-otp</code>	OTP-assisted password change.
POST	<code>/change-password/forgot/send-otp</code> , <code>/change-password/forgot</code>	Recovery-assisted password change.

Notifications — `/api/notifications`

Method	Path	Purpose
GET	<code>/</code>	Authenticated customer's notifications.
GET	<code>/unread-count</code>	Unread indicator.
PATCH	<code>/_{id}/read</code>	Mark an owned notification read.
PATCH	<code>/read-all</code>	Mark own notifications read.

Applicant/operator KYC — /api/station-manager

Method	Path	Purpose
GET	/reference-data	Allowed types/document requirements for application UI.
POST	/application	Multipart KYC/application create or update.
GET	/status/{referenceId}	Applicant tracking status; production design must strongly authenticate/verify and redact it.
GET	/application	Authenticated operator's own application.
GET	/application/files/{slotType}	Own standard KYC file.
GET	/application/business-documents/{documentType}/file	Own business-specific document.
POST	/session	Refresh/return an authenticated station-operator portal session linked to the approved application.

Admin KYC — /api/admin/station-manager-applications

Method	Path	Purpose
GET	/, /{id}	List and inspect applications.
GET	/ {id} /files/{slotType}	Review/download standard evidence.
GET	/ {id} /business-documents/{documentType}/file	Review/download business evidence.
POST	/ {id} /approve, / {id} /reject	Review decision.
POST	/ {id} /issue-credentials	Create/link operator access; must use secure one-time setup.

9. Complete API catalog — admin operations

All routes below use the `/api/admin` base and require exact authorization. Station operators must receive only linked-station data and actions; platform-wide controls belong to admins.

Area	Methods and paths	Purpose
Dashboard	GET <code>/dashboard</code>	Summary metrics appropriate to principal scope.
Stations	GET/POST <code>/stations</code> PUT/DELETE <code>/stations/{id}</code> PATCH <code>/stations/{id}/toggle</code>	Station list, create, edit, delete and enable/disable.
Charging points	GET <code>/charging-points/station/{stationId}</code> POST <code>/charging-points</code> PUT/DELETE <code>/charging-points/{id}</code> PATCH <code>/charging-points/{id}/status</code>	Inventory and operational state.
Pricing	GET/POST <code>/pricing</code> GET <code>/pricing/station/{stationId}</code> DELETE <code>/pricing/{id}</code>	Station pricing administration.
Bookings	GET <code>/bookings</code> , <code>/bookings/stats</code> , <code>/bookings/station/{stationId}</code> POST <code>/bookings/{id}/cancel</code> POST <code>/bookings/{id}/reschedule/approve</code> , <code>/reject</code>	Review, metrics, cancellation and reschedule decisions.
Customers	GET <code>/customers</code> PATCH <code>/customers/{id}/status</code>	Directory and activation/deactivation.
Sessions	GET <code>/sessions</code> POST <code>/sessions/{id}/end</code>	Monitor and force an authorized session end.
Bills	GET <code>/bills</code> , <code>/bills/{id}/invoice</code> POST <code>/bills/{id}/pay</code>	Financial operations, invoice and manual paid transition.
Revenue	GET <code>/revenue</code>	Revenue aggregates/reporting.
Audit	GET <code>/audit-logs</code>	Administrative event history.

Booking request and response shape

A create request carries `stationId` , optional `chargingPointId` , `startTime` , duration (1–60 minutes), optional point-type preference, and optional origin latitude/longitude. Origin coordinates select the dynamic/ETA path; a fixed booking can identify a point or ask the backend to auto-select.

A booking response can include the booking/reference, customer, station, point and vehicle details; start/end/duration; origin and last-known coordinates; last ping; predicted arrival and grace end; ETA/distance; proximity/assignment flags; virtual status; locked price; lifecycle status; cancellation/reschedule fields; and timestamps.

Station-manager application shape

Section	Representative fields
Personal	Full name, email, phone, date of birth, address, government ID type/number, ID reference and selfie.
Business	Business type/name/legal name, PAN, GST, registration, address, authorized signatory and required business documents.
Station/property	Name, address, city/state/pincode, latitude/longitude, owned/rented, property evidence, electricity consumer/bill, opening/closing hours and emergency contact.
Bank	Account holder, bank name, account number, IFSC and bank proof.
Capacity	Charger count, charger types, connectors, total capacity, manufacturers, installation and site photos.

10. Background jobs, notifications and documents

Job	Default cadence	Business effect
Session auto-completion	Initial delay about 1 second; check about every 1 second	Finds sessions reaching planned completion, ends them and drives settlement/state changes.
Wallet live monitor	Initial delay about 2 seconds; check about every 3 seconds	Calculates accumulated charging cost, debits wallet and reacts to insufficient funding.
Booking-start notification	Initial delay about 10 seconds; check about every 30 seconds	Finds upcoming/relevant bookings and creates reminder messages within a configurable lookback.
Booking expiry/no-show	Initial delay about 15 seconds; check about every 30 seconds	Expires missed bookings and releases/updates resources.

Cadence is controlled by environment variables. Very frequent schedules are useful for demos but require multi-instance coordination, distributed job claims, optimistic locking, idempotent handlers, capacity testing and sensible production intervals.

Notifications

- Persistent in-app notification documents support list, unread count, single read and read-all behavior.
- Booking reminders and lifecycle events can create customer notifications.
- The mobile project includes Expo Notifications configuration; backend push-token registration/delivery should be verified before claiming full remote push capability.
- Email covers registration/recovery OTPs, station-manager tracking/credentials, and invoice delivery.

Generated and downloaded documents

- OpenPDF generates invoice PDFs from billing data.
- Customer invoice and statement endpoints return downloadable files; mobile uses file-system support to save them.
- Admin KYC review retrieves binary evidence supplied by applicants.
- Sensitive documents and responses should use `Cache-Control: no-store`, safe normalized filenames, `X-Content-Type-Options: nosniff`, explicit attachment/sandbox policies and a separate untrusted-file origin where possible.

11. External integrations

Integration	Where used	Required production guarantees
MongoDB / Atlas	All backend persistence	TLS, least-privilege DB user, backups/restore tests, indexes, encryption, retention and safe migration.
Google Identity	Web/mobile sign-in; selected password-recovery UX	Backend verification of issuer, client/audience, signature, expiry and intended flow; never trust a client-supplied email alone.
Razorpay	Top-up orders, payment verification, mandate/auto-top-up and refund/withdrawal	Server-side signature verification, idempotency, stored provider IDs, unique constraints, reconciliation, webhook authenticity and no external call inside a rollbackable transaction without an outbox/state machine.
SMTP email	OTPs, tracking, credentials and invoices	Protected credentials, TLS, rate limits, non-enumerating responses, deliverability controls and no reusable plaintext passwords.
Maps and location	Mobile station map, nearby discovery, navigation/ETA and virtual bookings	Restricted platform/API keys, purpose-limited consent, minimum precision/retention and owner-only location access.
AWS S3	Static frontend deployment	Private deployment credentials/OIDC, correct cache headers, HTTPS CDN, SPA fallback and security headers.
EC2/systemd	Backend runtime deployment	Verified SSH host, patched host/JRE, non-root service, firewall, secret manager, health monitoring, logs and rollback.
Physical charger	Session command abstraction	STUB TODAY Real authenticated device protocol, command acknowledgements, signed meter values, replay protection, safety interlocks and reconciliation are required.

12. Configuration and local setup

Rule: values beginning with `VITE_` or `EXPO_PUBLIC_` are compiled into client applications and must be treated as public. Real secrets belong only in protected backend/deployment secret stores.

Backend configuration names

`SERVER_PORT`, `SERVER_ADDRESS`, `MONGODB_URI`, `MONGODB_DATABASE`, `JWT_SECRET`, `MAIL_HOST`, `MAIL_PORT`, `MAIL_USERNAME`, `MAIL_PASSWORD`, `MAIL_FROM`, `APP_BASE_URL`, `DEFAULT_RATE_PER_KWH`, `CHARGING_EFFICIENCY_FACTOR`, `SUPPORT_EMAIL`, `WEBSITE_URL`, `GOOGLE_CLIENT_IDS`, `GOOGLE_MAPS_API_KEY`, `RAZORPAY_KEY_ID`, `RAZORPAY_KEY_SECRET`, `RAZORPAY_CURRENCY`, `RAZORPAY_BUSINESS_NAME`, `RAZORPAY_MINIMUM_AMOUNT`.

Scheduling controls include `BOOKING_START_NOTIFICATION_INITIAL_DELAY_MS`, `BOOKING_START_NOTIFICATION_CHECK_MS`, `BOOKING_START_NOTIFICATION_LOOKBACK_MINUTES`, `WALLET_MONITOR_INITIAL_DELAY_MS`, `WALLET_MONITOR_CHECK_MS`, `BOOKING_EXPIRY_INITIAL_DELAY_MS`, `BOOKING_EXPIRY_CHECK_MS`, `SESSION_AUTO_COMPLETE_INITIAL_DELAY_MS`, and `SESSION_AUTO_COMPLETE_CHECK_MS`.

Web configuration names

`VITE_API_BASE_URL`, `VITE_API_TIMEOUT_MS`, `VITE_API_PROXY_TARGET`, `VITE_USE_REMOTE_API`, and `VITE_GOOGLE_CLIENT_ID`. Production must use an HTTPS API URL.

Mobile configuration names

The app/scripts use API port/base/LAN/timeout and Google client configuration, including `EXPO_PUBLIC_API_PORT`, `EXPO_PUBLIC_API_BASE_URL`, `EXPO_PUBLIC_API_LAN_BASE_URL`, `EXPO_PUBLIC_API_TIMEOUT_MS`, `EXPO_PUBLIC_OTP_TIMEOUT_MS`, `EXPO_PUBLIC_PAYMENT_TIMEOUT_MS`, `EXPO_PUBLIC_GOOGLE_CLIENT_ID`, `EXPO_PUBLIC_GOOGLE_ANDROID_CLIENT_ID`, `EXPO_PUBLIC_METRO_HOST`, `REACT_NATIVE_PACKAGER_HOSTNAME`, `NODE_ENV`, and `EXPO_OFFLINE`. Exact supported names should be confirmed in `src/api/client.js`, app configuration and wireless scripts before deployment.

Typical local run

1. MongoDB

Run local MongoDB or supply a protected Atlas URI and database name.

2. Backend

```
cd plugin-backend
mvn clean install
mvn spring-boot:run
```

Default API: port 8091.

3. Website

```
cd plugin-frontend
npm install
npm run dev
```

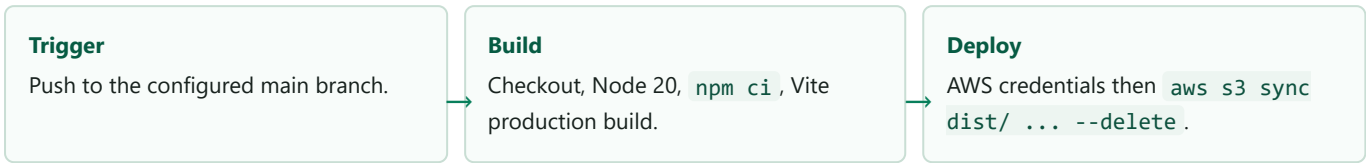
Default Vite: port 5173.

4. Mobile

`cd plugin_app` then `npm install` and `npm start`. For a physical Android device, the repository includes wireless/LAN setup, install, update and offline release scripts. The device must reach the backend address selected by the client.

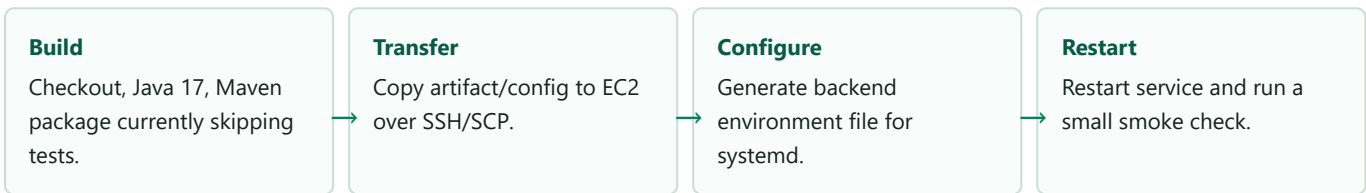
13. Build and deployment

Website pipeline



A production serving layer should add HTTPS/CDN, SPA fallback, cache strategy and security headers (CSP, frame ancestors, HSTS, nosniff, referrer and permissions policy).

Backend pipeline



The backend also has a multi-stage Dockerfile: Maven/Temurin build stage and a Java 17 runtime stage. The current build skips tests.

Mobile build

The Android project is generated/maintained alongside Expo. Scripts cover Expo start, wireless setup, debug installation, Android run, offline release build and app update. No complete hosted mobile release pipeline or app-store publishing workflow was found in the repository.

Deployment hardening required: current workflows expose broad production secrets too early, use movable action tags, and the backend transfer path disables SSH host verification. Split secretless build from protected deployment, pin action SHAs, set explicit minimum permissions, use short-lived AWS OIDC/instance identity, keep secrets step-scoped, verify the host key, sign/verify artifacts, and use a protected environment with approval and rollback.

14. Testing, logging and observability

Current executable checks

- Website provides `npm run build`; no dedicated lint or test script is declared in its package manifest.
- Mobile provides Expo/Android build and run scripts; no dedicated automated unit/e2e test script is declared.
- Backend uses Spring Boot test dependencies. The current test run compiled and executed 9 tests, with 3 failures in `BookingOverlapTest`. Only two backend test classes were observed during the security review.
- There is not yet adequate automated coverage for authorization/BOLA, session ownership, wallet/payment idempotency, concurrent schedulers, KYC validation, upload safety or security headers.

Production observability needs

- Structured logs with request/correlation ID, principal ID, safe event name and latency—never OTPs, JWTs, passwords, bank/KYC data or full provider payloads.
- Metrics for API error/latency rates, login/OTP abuse, booking conflicts, active sessions, scheduler lag, wallet discrepancies, provider reconciliation, mail failures and file-upload rejection.
- Alerts for payment mismatch, repeated authorization denials, abnormal KYC enumeration/upload volume, session completion failure, database connectivity, expired certificates and deployment smoke-test failure.
- Audit events for administrative role/status/price/station/session/bill/KYC actions with actor, target, before/after summary, reason and timestamp.
- Health endpoints that separate liveness from readiness; protect detailed actuator output from the public internet.

Minimum release test layers

1. Unit tests for calculations, lifecycle transitions and validation.
2. Repository/service integration tests with MongoDB and unique indexes.
3. API authorization matrix for public/customer/operator/admin and cross-user IDs.
4. Concurrency and fault-injection tests for booking allocation, session end, wallet debit, top-up, refund and schedulers.
5. Web/mobile end-to-end happy paths plus denial/error/retry/offline paths.
6. SAST, secret scanning, dependency/OSV auditing, container/mobile artifact scanning and production security-header checks.
7. Payment-provider test-mode reconciliation and verified webhook/idempotency scenarios.
8. Backup restore, deployment rollback and mobile upgrade tests.

15. Security and launch status

Current decision: NO-GO for a public production launch. A repository-level review identified multiple confirmed critical/high issues. The detailed evidence and regression plan are in [security-audit/REPORT.md](#) , [COVERAGE.md](#) , and [SECURITY-TEST-PLAN.md](#) . This summary exists so a new collaborator understands the product's present boundary.

Current risk	Before remediation	Required end state
Public KYC tracking disclosure and overwrite	An attacker can enumerate predictable references, retrieve highly sensitive personal/bank/KYC data, learn the victim email, and resubmit/replace an existing application.	Authenticated or high-entropy one-time tracking proof, applicant verification, non-enumerating responses, redaction, rate limits, immutable review transitions and owner-bound updates.
Hostile KYC file preview	Attacker-controlled content/type/name can reach an unsandboxed admin preview and potentially execute script in an admin origin, stealing a localStorage JWT.	Server magic-byte allowlist, size/count quotas, AV/CDR, reject SVG/HTML/polyglots, isolate file origin, force safe headers/downloads, sandbox previews, and remove bearer tokens from script-accessible storage.
Cross-user object access	Raw booking, session, bill and notification IDs can expose another customer's location, vehicle or financial data; session end can affect another user.	Principal-bound repository queries and explicit authorization on every read/mutation, with automated horizontal/vertical access tests.
Booking/location allocation bypass	A non-virtual booking can misuse location update to move timing and claim a point; dynamic allocation may ignore fixed overlap.	Strict virtual-state transition checks plus atomic interval-aware point allocation and concurrency tests.
Wallet/session/payment races	Crashes, concurrent jobs or retries can duplicate debits/refunds/orders or revert terminal state; external success can diverge from rolled-back Mongo state.	Unique idempotency keys, stored response replay, optimistic versioning/CAS, distributed job claims, provider state machine/outbox, reconciliation and fault tests.
Mobile insecure transport/retries/signing	Bearer data can fall back to HTTP; short timeout retries duplicate mutations; release uses debug signing material.	HTTPS-only with network security policy/pinning as appropriate, safe retry policy plus server idempotency, and protected production signing/release provenance.
Default/previous credentials	A public JWT fallback is accepted and matches a local environment; a historical mail credential was committed.	Fail startup without a strong managed JWT secret, rotate/revoke all exposed credentials/tokens, shorten token lifetime and purge history where warranted.
Role/station scope confusion	Operator can pass broad web admin guard and may probe platform-wide IDs/APIs.	Exact role checks plus station-linked ABAC in backend queries; frontend route definitions mirror intended access but are not relied upon.
CI/CD secret exposure	Build-time code/actions receive production secrets; SSH host authenticity is not verified.	Secretless reproducible build, pinned actions, short-lived least privilege, verified host/artifact, protected deploy environment and rollback.
Charger/meter trust gap	Billing uses theoretical power/time and stop command only logs; physical charging may continue or charges may not reflect a trusted meter.	Authenticated charger protocol and meter data, device acknowledgement, safety controls, reconciliation and independently tested settlement.

Security architecture target

- Short-lived, correctly validated access tokens; secure rotation/revocation. For the browser, prefer Secure HttpOnly SameSite cookies with deliberate CSRF protection, or another design that prevents script-

readable token theft.

- Central deny-by-default authorization policy with ownership and operator station scope in the data query itself.
- HTTPS/HSTS everywhere; restricted CORS; CSP and safe framing; no sensitive caching; safe error envelopes.
- Strict server-side validation, quotas and lifecycle controls for all user input, especially multipart KYC and financial operations.
- Idempotent commands and explicit state machines for money, sessions, allocation and external providers.
- Security gates in CI and launch criteria defined by passing regression tests—not only manual review.

Additional current implementation constraints

- JWT claims remain trusted until their roughly 24-hour expiry; there is no complete refresh-token/revocation flow or per-request account-role revalidation.
- OTPs are stored as directly comparable values and lack a complete attempt counter, resend cooldown and account/IP rate-limiting policy.
- The public station-list implementation should be reviewed because it can return inactive stations rather than applying `active=true` consistently.
- Top-up verification validates a signature but still needs provider-side amount, currency, capture-state, order-owner and reconciliation checks before crediting value.
- MongoDB transaction behavior requires a transaction-capable deployment topology; in-process `synchronized` locks do not coordinate multiple API instances.
- There is no application-wide rate-limiting middleware or complete payment webhook processing in the current backend.
- The Docker image build skips tests and does not declare a dedicated non-root runtime user or health check.

16. Implemented, partial and roadmap boundaries

Status	Capabilities
IMPLEMENTED IN CODE	Email/OTP and Google auth; station catalog/search/details; customer vehicles; fixed and virtual/ETA bookings; booking actions; session start/end/history; calculated energy/cost; bills/invoice/statement; wallet/ledger/top-up/mandate/withdrawal; notifications; profile/password/deletion; station-manager KYC; admin/operator console; MongoDB persistence; web and Android/Expo clients; AWS/EC2 deployment workflows.
PARTIAL / NOT PRODUCTION COMPLETE	Physical charger commands and trusted metering; remote push delivery proof; mobile release signing; production transport/headers; payment/session idempotency and distributed job safety; comprehensive automated tests; hardened KYC file pipeline; exact operator scoping; production secret/deployment model.
PLANNING IDEAS ONLY	Battery-aware route planning, virtual queues, availability forecasting, peer handoff, off-peak smart scheduling, idle-fee automation, 80% automatic stop, clean-bay rewards, eco-charge rewards, amenity discounts and tipping.

Source hierarchy for future discussions

1. **Current repository code and runtime configuration** — authoritative for what executes.
2. **This guide** — authoritative map of the examined snapshot.
3. **Current security audit artifacts** — authoritative record of validated security findings at the snapshot.
4. **README** — quick setup/overview, sometimes less detailed than code.
5. **PLUGIN SRS** — older baseline; useful for intent, but some database/mobile/payment scope is outdated.
6. **plugin_app_future.pdf** — roadmap ideas, never evidence of current implementation.

17. Glossary

Term	Meaning in this project
Fixed booking	A reservation for a known physical charging point and time range.
Virtual / ETA booking	A reservation without an initial physical point; location and point preference help allocate near arrival.
Proximity lock	Virtual-booking state indicating the user is close enough for physical allocation/locking behavior.
Pricing snapshot	Rate copied into a booking so later station price edits do not silently change it.
Charging point	Physical station connector/charger resource with type, power and operational status.
Session	Charging lifecycle started from a booking and completed/aborted into billing history.
Wallet ledger	History of credits, debits, refunds and reversals behind the displayed balance.
Mandate	Payment-provider authorization used for automatic top-up.
KYC	Identity/business/property/bank verification for a station-manager applicant.
BOLA / IDOR	Broken object-level authorization: using an object ID to read/change another user's record.
ABAC	Attribute-based access control, such as allowing an operator only when the target belongs to their linked station.
Idempotency key	Unique client command ID that makes retries return the original result instead of executing money/resource mutation again.
Outbox/state machine	Pattern for reliably coordinating database state with external provider calls and retries.
Source of truth	The authoritative state. Backend/database/provider reconciliation—not client UI—is authoritative for security, price and payment.

Key code entry points

Area	Entry points
Web routes	<code>plugin-frontend/src/App.jsx</code>
Web authentication/network	<code>plugin-frontend/src/context/AuthContext.jsx</code> , <code>plugin-frontend/src/api/axios.js</code>
Mobile orchestration/network	<code>plugin_app/App.js</code> , <code>plugin_app/src/api/client.js</code>
Backend configuration/security	<code>plugin-backend/src/main/resources/application.yml</code> , backend security configuration and JWT classes
Backend API	<code>plugin-backend/src/main/java/com/plugin/controller/</code>
Business logic	<code>plugin-backend/src/main/java/com/plugin/service/</code>
Domain/persistence	<code>plugin-backend/src/main/java/com/plugin/entity/</code> , <code>repository/</code> , <code>enums/</code>
Delivery	<code>.github/workflows/</code> , backend <code>Dockerfile</code> , mobile <code>android/</code> and <code>scripts/</code>

18. AI handoff checklist

When asking ChatGPT to modify PLUGIN, require it to answer these questions before or alongside the implementation:

1. Is this current functionality, a partial integration, or a roadmap request?
2. Which application(s) change: backend, website, mobile, deployment, or data migration?
3. Which users and exact roles can perform the action?
4. What object ownership or station scope must the backend enforce?
5. Which API request/response contracts and existing clients are affected?
6. Which entities, indexes, lifecycle states and historical records are affected?
7. Can the action be retried, run concurrently, or partially fail? What is the idempotency/rollback/reconciliation design?
8. Does it handle money, precise location, credentials, identity/KYC, uploaded files or other sensitive data?
9. What server-side validation and rate/size/time limits are required?
10. What web/mobile error, loading, offline and stale-state behavior is required?
11. What logs, metrics, audit events and alerts will prove it works safely?
12. Which unit, integration, authorization, concurrency, end-to-end and security tests must pass?
13. Does deployment need new environment names, secret-store entries, database indexes/migration, mobile permissions or rollback instructions?
14. Does the change conflict with any finding in `security-audit/REPORT.md` or need a security regression test?

Reusable request template

"Using the attached PLUGIN guide and the current repository, implement [goal]. First identify affected roles, routes/screens, API endpoints, services, entities, states, integrations and security boundaries. Preserve existing user-owned changes. Do not trust client-side role, price, ID, location or payment claims. Make retry/concurrency behavior explicit. Update web and mobile clients when their contract changes. Add proportionate tests, run the relevant builds/tests, and finish with files changed, before/after behavior, remaining risks and deployment steps."

End of guide. This document intentionally contains architecture and behavior, but no secret values. Generated from the repository snapshot at commit `0064b3dd93e1` on 9 August 2026.